

35.6. BONHOMME, de son fichier à l'écran : `sprite16()`

Le BONHOMME de `perso_BONHOMME.cpp` devient un lutin : `sprite16()` le pose au pixel près, et `principal.cpp` n'a qu'une ligne pour l'avoir.

1. Ce qu'il faut comprendre

Ce cours fait voyager un personnage entre les fichiers, une étape à la fois : il est rangé dans `perso_BONHOMME.cpp`, on l'appelle depuis un **autre fichier**, on en met **deux** à l'écran, on le **fait bouger**, et on le **verrouille**.

Ce qui est nouveau ici : le BONHOMME du 35.1 (dans `perso_BONHOMME.cpp`, sans changement) n'est plus posé en quatre cases : `sprite16(0, 72, 64, BONHOMME)` en fait un **lutin** de 16×16 — les lutins 0, 1, 2 et 3, en carré, par-dessus le fond.

principal.cpp n'a qu'une ligne pour l'avoir : `#include "personnages.cpp"`, la liste. Le dessin et ses seize rangées sont dans `perso_BONHOMME.cpp` ; l'autorisation `#include <Perso>`, une fois, dans la liste. On voit ce que fait le jeu, sans les rangées du dessin au milieu.

Rappel : `#include "personnages.cpp"` colle la liste à cet endroit avant de compiler, et la liste colle `perso_BONHOMME.cpp` (des guillemets : un fichier à toi). `#include <sprite16>` **demande** une fonction de la console (des chevrons).

Dans l'atelier : « + perso » de « 🧱 Les tuiles » donne tout seul son fichier à chaque nouveau personnage ; pour ceux qui existent déjà, « 📁 Un fichier par personnage » dans « 🏠 Tout le jeu ».

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de \$16 \times 16\$](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **336 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage — la mise en route de la console
main()	106 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles	64 o	← CopierTuiles — 0 lettre sur 43, et 4 tuiles dessinées

```
DonneesTransfert      10 o ← InstallerTransfert – la petite routine qui copie les lutins
Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
  sprite()   × 4   96 o sur place, dans main()
  image()    × 1    6 o sur place, dans main() + AttendreImage (28 o)
  sprite16() × 1    0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// BONHOMME vient de perso_BONHOMME.cpp ; ici, on s'en sert.

#include <sprite16>          // sprite16() : quatre lutins en carré
#include "personnages.cpp"   // la liste : BONHOMME
```

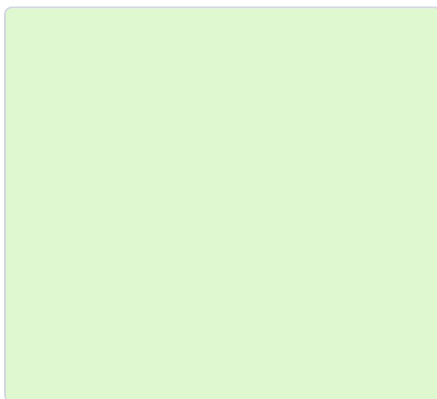
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  sprite16(0, 72, 64, BONHOMME); // les lutins 0 à 3, coin haut-gauche en (72, 64)

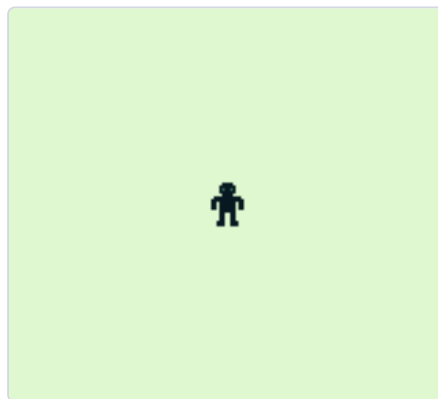
  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le bonhomme au milieu de l'écran, en lutin.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin 0 est en (72, 64) (72, 64)

✓ les quatre lutins font un carré de 16

✓ avec les quatre tuiles de BONHOMME

Ensuite : **35.7. Un nouveau fichier qui appelle BONHOMME : heros.cpp** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++